Java VS Javascript

Java is a programming language, in which the code will get executed sequentially.	In Javascript the program is event driven, the execution of code is dependent on user action
Java is statically type, means the data type of variable is decided at compile time	In javascript type of variables is decided at run time, based on the value you have assigned to the variable
Java is Object oriented.	Javascript is object based.

To declare a variable in javascript, you can use keyword let, var and const

let	var	const
duplicate declaration is not allowed	duplicate declaration is allowed	duplicate declaration is not allowed
let is a blocked scope variable	var is global or function scope variable	const is blocked scope
hoisting of variable declaration is not allowed	hoisting of variable declaration is allowed	hoisting of variable declaration is not allowed
let v;	var v;	const c=34; Initialization of variable is needed, at the time of declaration.

In client side javascript

1. Client side data validation is possible
2. Give dynamic look and feel to the page
3. AJAX(Asynchronous Javascript and XML) requests can be send by javascript to webservice, by using XmlHttpRequest(XHR) object

The Javascript code can be executed with the help of HTML, and is called as Client side javascript code.

Or it can be executed with the help of Nodejs, which is server side code.

To display popup boxes in javascript, there are 3 types of boxes, and all these functions are of window objects

Popup boxes	
alert	A box with only one ok button, useful to show messages to user
confirm	A box with 2 buttons ok and cancel, if user clicks on ok button then it returns true, otherwise, if user clicks on cancel button, then it returns false
prompt	A box with a text box, useful to accept one input from user

To display ouput to user in browser window, document.write(“Hello world!!”)

To display messages for testing purpose the use console.log(“Test messages”);

Ways to write functions

const f3=(x,y=3,...s)=>{//rest parameter
document.write("in f3",x,y,"
")
}
f1(1,2,3,4)
f2(1,2)
f3(1,2,3,4)
	function using => operator, you may assign default values to parameters
	It does not have arguments array

function f1(x,...arr){
document.write("in f1",x,"
");
document.write("test:"
+arr.length,arguments[0]);
}
	... rest parameters are useful for variable number of arguments

const f2=function(x,y){
document.write("in f2",x,y,"
");
}
//var f2=45;
const f4=(x,y=45,z=4)=>{
console.log(x,y,z);
 	
}
	better to use const keyword while declaring function, so that definition will not get overwritten

function f1(){
function f2(){
}
}
	Closure function→the function which has access to its parent scope, is called as closure function

(function(x,y){ document.write("in function f1",x,y) })(12,13)	Self calling function, If you want to execute the function definition only once, then use self calling function Usually these functions are used in JQuery plugins

functions in javascript for typecasting

parseInt	to convert the string into integer
parseFloat	To convert the string into float

The tags which has innerHTML property-→ div, span, p, a,h1,h2....

The tags which are not used to accept data from user via form, use innerHTML property

but the tags which are used in the form to accept data from user has value property

functions on numeric data

isNaN(data)	if the data contains atleast one character, then it returns true, otherwise it returns flase
isFinite(data)	if the data contains all digits between 0-9, then it returns true, otherwise it returns false

Validate data in javascript

validatedropdown	In dropdown box it stores property by name selectedIndex, it stores the index position of the selected option
	
	It also stores a array by name options, hence to find the value of selected dropdown box in javascript
	
	//get reference of dropdown box centers
	var c=document.getElementById(id);
	//retrieve the value of index of the selected option
	var pos=c.selectedIndex;
	//alert(c.selectedIndex)
	//retrieve the value of selected option
	var val=c.options[pos].value;

validateradio	to find which radio button is checked we use checked property, In radio button we can select only one option, so as soon as we get the checked radio button we may break the loop
	const validateradio=(nm,errid)=>{
	var arr=document.getElementsByName("gender");
	for(var i=0;i<arr.length;i++){
	if(arr[i].checked){
	document.getElementById(errid).innerHTML="";
	return true;
	}
	}
	document.getElementById(errid).innerHTML="pls select one choice"
	return false;
	}

validatecheckbox	to find which check is checked, we use checked property, checkboxes are used to select multiple options, we use the checked property to find which checkboxes are checked
	const validatecheckbox=(nm,errid)=>{
	var subarr=document.getElementsByName(nm);
	var cnt=0;
	var str="";
	for(var i=0;i<subarr.length;i++){
	if(subarr[i].checked){
	cnt++;
	str=str+subarr[i].value+",";
	}
	/*if(cnt>=2){
	break;
	}*/
	}
	if(cnt>=2){

	//clear the message
	document.getElementById(errid).innerHTML="";
	alert(str);
	return true;
	}
	//display error message
	document.getElementById(errid).innerHTML="pls select minimum 2 subjects"
	return false;
	}

validatefile	to validate file size, we may use size property
	
	function validateSize(){
	alert("in validatesize")
	var f=document.getElementById("files");
	//f.files[0].size it will give size in bytes
	alert(f.files[0].size/1024+"KB");
	if(f.files[0].size/1024 < 10){
	alert("in if")
	document.getElementById("errfile").innerHTML="";
	//return true;
	}
	document.getElementById("errfile").innerHTML="file size exceeds the limit 10KB"
	//return false;
	}

Promise object is always in one of the 3 states

1. Pending --→ when the asynchronous function is executing, the it will be in pending state
2. Fulfil /Resolved---→ if asynchronous function finishes successfully then it will be in resolved or fulfil state
   In resolved state it executes then function
3. Rejected-→ if asynchronous function finishes with error, then it will be in rejected state
   In rehjected state it executes catch function

operators

Arithmetic operators	+, -, *, / , % , **(exponential) 3**3=27 (3 raise to 3)

relational operator	>, <, <=,>=, == , ===, !=, !==
	
	=== → strict checking
	10===”10” false //it check value as well as data type
	10===10 true
	10==”10” true
	10===10 true

logical operators	&& , || , !

Functions

a.toFixed(num)	to display the number upto 2 decimal places a=3.3333333 a.toFixed(2)= 3.33
Math.round(num)	it will round the number Math.round(3.456778)= 3 Math.round(3.6789)=4

String functions

s.trim()	to remove leading and trailing spaces	s=" aaaaa " s.trim() aaaaa
s.length	to find length of the string	s.length
s.charAt(i)	to find the character at specific index position	s="testing" s.charAt(4)=i
s.concat(str1,str2,.....)	to concat multiple strings	s="taste" s.concat("is", "good") taste is good

s.indexOf(str)	to find the position of first occurrence of the given str, indexing starts with 0	s="Hello World!!"
		s.indeXOf("Hello") //0
		s.indexOf("World) //6

s.split(delimiter)	to break the string into array of strings at delimiter position	s="rain in spain, is pain" s.split(" ") ["rain", "in","spain,","is","pain"] s.split(",") ["rain in spain"," is pain"]
s.toUpperCase()	convert the string into uppercase	s="abcd" s.toUpperCase() ABCD
s.toLowerCase()	convert the string into lowercase	s="ABcd" s.toLowerCase() abcd

s.slice(startIndex[,endIndex])	it retrieves the portion of the string from startIndex to endIndex, endIndex is excluded	s="Hello world!!"
		s.slice(0,1) //H
		
		s.slice(4) //o world!!

s.includes(substr)	it returns true, if substring exists, false otherwise	s="Hello world!!"
		s.includes("ell") //true
		
		s.includes("xxx") //false

using regular expression

s.match(regular expression)	it checks the specified value in the string, and return all occurences	s="rain in SPAIN is pain" pattern=/a.*?n/ s.match(pattern) //2

		pattern=/a.*?n/i -→ values enclosed in / / is treated as regular expression i is flag for ignore index g is a flag for global s.match(pattern) //2
s.replace(regexpression, newstring)	It will search the given pattern and replaces it with the newstring	s=”To be or not to be” s.replace(/be/,”exists”) –>replace first occurence s.replace(/be/g,”exists”)-> flag g, will replace all occurences

s.search(regexpression)	it checks the specified pattern in the string, and return the position of first occurrence	var test=”testing: 1, 2, 3”
		var pattern= /\d+/g
		s.search(pattern) //9
		s.match(pattern) // [“1”,” 2”,”3”]
		s.replace(pattern,”#”) // testing: #, #,#